Applied ML

24-02-26

Data transformation

There are various techniques using which raw data is transformed and made suitable for ML training

Log transformation

- applying log function on the data

$$d = [1, 2, 3, 4, 5, 6, 10, \underbrace{100, 200, 1000}_{\text{outliers}}]$$

$\downarrow$ log transformation

$$\log_{10}(d) = [\log(1), \log(2), \log(3) \ldots\ldots \log(200), \log(1000)]$$
$$= [0, 0.04, 0.5, 0.6, 0.8, 0.9, 1, 2, 2.4, 3]$$

$\log_{10}(10) = 1$ $\qquad$ $\log_{10}(1000) = 3$

$\log_{10}(100) = 2$ $\qquad$ $\log_{10}(1) = 0$

- After log transformation, difference betn outliers and rest of the datapoints reduces. This reduces the effect of outliers. That makes ML training smooth.

ML models work great when data used for their training is normally distributed and log transform converts skewed data into more normally distributed data.

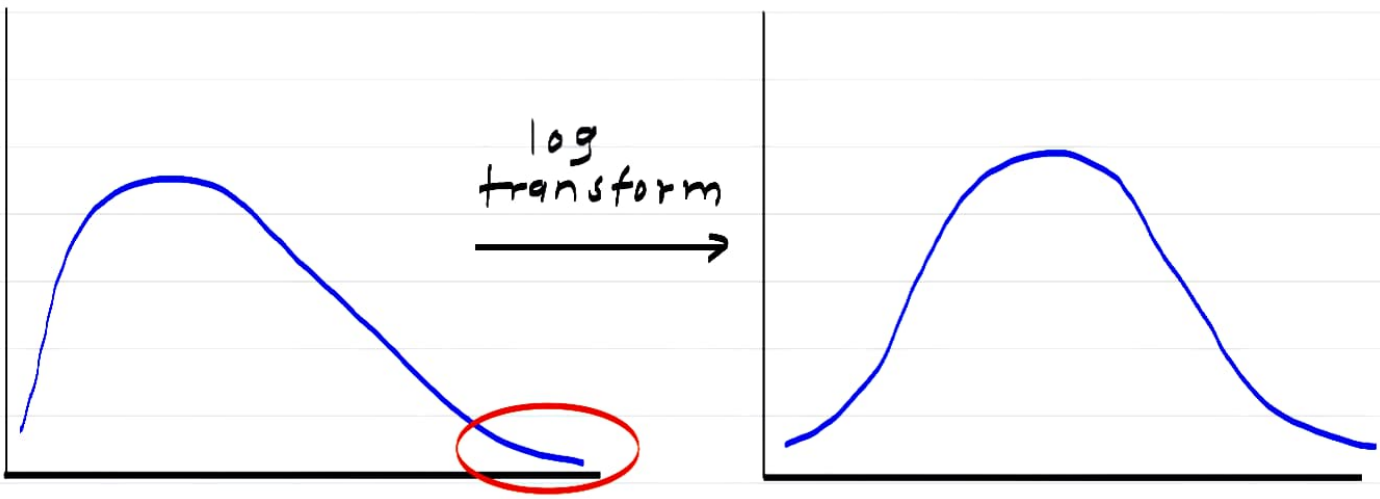
x_1	x_2	x_3	y
1	40	0.5	100
2	30	0.7	5000
3	20	0.8	4000
1000	21	1	200
4000	25	1.2	6000
⋮			⋮

x_1, x_2, x_3 → [ML model] → y

lets say x_1 and y are having outliers that we can't remove

$\log(x_1)$	x_2	x_3	$\log(y)$
⋮			2 ⋮

$\log(x_1)$, x_2, x_3 → [ML model] → $\log(y)$



Feature Scaling

area	rooms	price
1200	2	70
800	2	80
900	2	60
5000	5	90
2400	3	125
2700	3	120
⋮	⋮	⋮

features

area has 4 digit scale
rooms has 1 digit scale

Difference in feature scales is not good for ML training as model gives more importance to feature with higher scale.
We don't want any such bias in ML training

Hence, we apply feature scaling to match the scales of features.

We also apply this scaling on target variable.

1) Min-max scaling

2) Standardization ✓

Min-max scaling

x_1	x_1'	y	x_2	x_2'
1	$\frac{1-1}{5-1} = 0$		100	$\frac{100-100}{600-100} = 0$
2	$2-1/4 = 1/4$		200	$100/500 = 1/5$
3	$3-1/4 = 2/4$		300	$200/500 = 2/5$
4	$4-1/4 = 3/4$		450	$350/500 = 3.5/5$
5	$5-1/4 = 4/4 = 1$		600	$500/500 = 1$
↓			↓	
min = 1			min = 100	
max = 5			max = 600	

$$x' = \frac{x - \min}{\max - \min}$$

$x_1' = [0, 1/4, 2/4, 3/4, 4/4]$
$x_2' = [0, 1/5, 2/5, 3.5/5, 5/5]$
} scales are same

$$\begin{matrix} x_1' \\ x_2' \end{matrix} \longrightarrow \boxed{\text{model}} \longrightarrow y$$

After min-max scaling, all values lie between 0-1

Standardization

x_1 ⋮ μ_1 σ_1

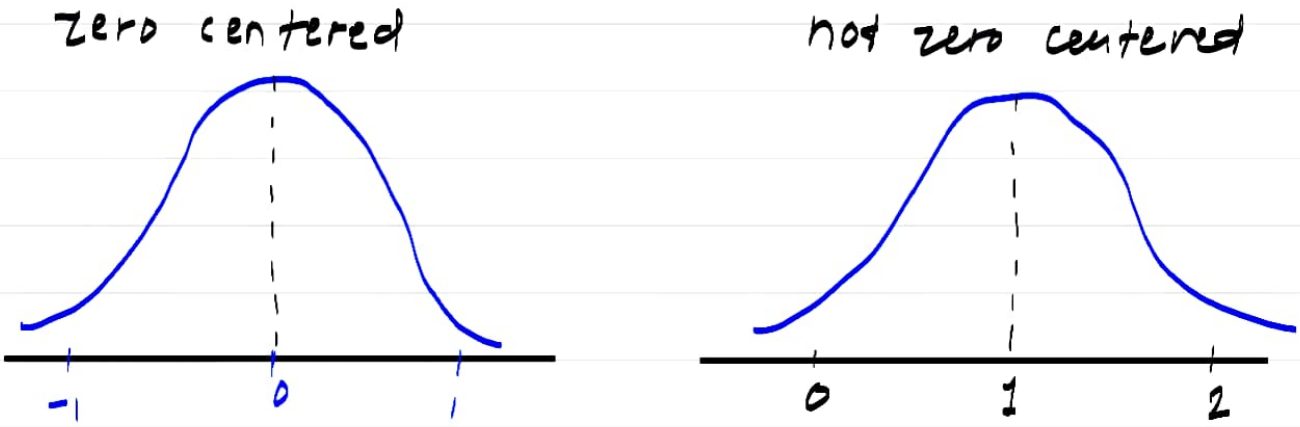
x_2 ⋮ μ_2 σ_2

$$\boxed{x' = \frac{x - \mu}{\sigma}}$$

$$x_1' = \frac{x_1 - \mu_1}{\sigma_1} \qquad x_2' = \frac{x_2 - \mu_2}{\sigma_2}$$

ML model training happens very smoothly when data is <u>zero centered</u>.

Zero centered → mean of the data lies near zero.



Feature Encoding

- Most of the ML models work only on numerical data
- hence, categorical data needs to be converted to numerical data before using it in model training

area, rooms, city → [model] → price

area ↓ num, rooms ↓ num, city ↓ categorical

Hyd, Delhi, Pune, Blr
↓ ↓ ↓ ↓
1 2 3 4

} categories converted to numbers

Sub types of categorical data

nominal - no ordering among categories
eg. hyd, delhi, pune, blr red, blue, green

ordinal - specific order among categories
eg. small, med, large, / low, med, high

binary - 2 opposite categories
eg. yes/no true/false, on/off

There are multiple feature encoding techniques

One hot encoding

- usually used for binary or nominal data

eg.

area	rooms	city	price
1200	2	Hyd	
1300	2	Delhi	
1500	2	Pune	
⋮		⋮	

needs encoding

1) find all unique categories in given column

df['city'].unique() → [Hyd, Delhi, Pune]

2) add a new column in original dataset for every unique category

area	rooms	city	price
1200	2	Hyd	10
1300	2	Delhi	11
1500	2	Pune	12
⋮	⋮	⋮	⋮

3 features

area	rooms	Hyd	Delhi	Pune	Price
1200	2	1	0	0	10
1300	2	0	1	0	11
1500	2	0	0	1	12
⋮	⋮	⋮	⋮	⋮	⋮

5 features

- when number of unique categories is very large then one hot encoding is not feasible as it will add too many new columns.

Label Encoding

- usually preferred for ordinal data
  - as it maintains order among categories

- also used for nominal data when number of unique categories is too large

eg.

area	room	city	type	price
1200	2	Hyd	basic	
1300	2	Pune	luxury	
⋮	⋮	Blr	premium	
		Chn	basic	
		Kol	luxury	
		Nag	⋮	
		Mum	⋮	

nominal → too many unq categories

ordinal → label encoding

basic < premium < luxury

1 < 2 < 3

we'll use label encoding for 'city' and 'type'

city - too many unique categories
type - ordinal data

area	rooms	city	type	price
		1	1	⋮
		2	2	⋮
		3	3	⋮
		4	2	⋮
		5	1	
		6	3	
		⋮	⋮	

encoded features

Binary Encoding

- only used for binary data

area	rooms	resale	price
1200	1	yes	⋮
1300	2	no	⋮
⋮	⋮	yes	⋮
⋮	⋮	no	⋮

resale
1
0
1
0
⋮

yes = 1
no = 0

Feature Engineering

- Sometimes, not all of the features in data are important for given use case
  in such cases we try to find the important features using feature engineering.

- Less important/ useless features are removed from ML training.